

Evictable Vector Commitments: Verifiable LLM Inference under KV-Cache Eviction

Anonymous Submission

Abstract. Verifiable LLM inference lets a client check that a model was executed faithfully on its input. Existing schemes commit to the entire retained activation state and therefore cannot account for *KV-cache eviction*: the permanent deletion of token key/value states performed by streaming cache compressors such as StreamingLLM, H2O, and SnapKV. We introduce *evictable vector commitments* (EvictVC), a commitment primitive whose committed positions can be permanently evicted under a public eviction policy while a tamper-evident history of every deletion is maintained. We formalize the primitive with a syntax and three game-based security notions—eviction soundness, policy-binding, and history tamper-evidence—and give a construction from KZG polynomial commitments in which an eviction is a single group operation independent of the cache size, an audit proof for an arbitrary set of positions is a single group element, and all opening proofs are maintainable in quasi-linear time via the Feist–Khovratovich technique. We prove security under the n -SDH assumption and the collision resistance of a hash function. A prototype over BLS12-381 confirms the asymptotics: eviction and commitment updates cost $\approx 135\ \mu\text{s}$ irrespective of cache length up to 2^{16} , verification is two pairings ($\approx 2.6\ \text{ms}$), and audit proofs are 48 bytes regardless of how many positions are audited.

Keywords: Vector commitments · KZG polynomial commitments · Verifiable computation · Machine-learning inference · KV-cache compression

1 Introduction

Delegated large-language-model inference increasingly runs on servers a client does not control. *Verifiable inference* addresses the resulting trust gap by having the server commit to its computation and prove that the claimed output follows from a faithful execution. A common ingredient is a commitment to the model’s intermediate state, against which the server later opens individual values to answer audit challenges.

A central object of that state is the *key/value (KV) cache*: the stored attention keys and values for every past token. The cache grows linearly with context length, so production systems compress it. Streaming compressors such as StreamingLLM, H2O, and SnapKV do so by *permanently evicting* the key/value states of tokens deemed unimportant under an attention-based policy. Eviction is destructive: the evicted state is gone and can never be opened again.

This destructiveness breaks existing verifiable-inference commitments, which assume the committed state remains fully present and openable. Three problems arise simultaneously. First, the commitment must be able to *shrink*: an evicted position can no longer be claimed to hold a value. Second, eviction must be *policy-bound*: a malicious server must not be able to drop an inconvenient (e.g. high-attention) position while claiming it followed the compressor’s policy. Third, the server must not be able to *rewrite history*: the record of which positions were evicted, and with what values, must be immutable so that an auditor can reconstruct and check the entire compression trajectory.

We resolve all three with *evictable vector commitments*. An EvictVC commits jointly to a value vector and a per-position score (attention) vector. It supports the usual openings

and homomorphic updates, and adds an **Evict** operation that (i) removes a position from the live value commitment in a single group operation, (ii) is accepted only if the position’s committed score satisfies the public eviction policy, and (iii) appends the removed record to a tamper-evident hash-chain history. Audits remain cheap: a proof for an arbitrary set of live positions is a single group element, and all opening proofs are maintainable in quasi-linear total time.

Contributions.

- **A model.** We give a syntax for evictable vector commitments and three game-based security notions—*eviction soundness*, *policy-binding*, and *history tamper-evidence* (Sections 4–5).
- **A construction.** We instantiate the primitive from KZG polynomial commitments over roots of unity (Π_{KZG} , Sections 6–7): eviction and updates are $O(1)$, audit proofs for any position set are a single group element, and all opening proofs are produced in $O(n \log n)$ via the Feist–Khovratovich technique.
- **Security proofs.** We give tight reductions: eviction soundness and policy-binding reduce to position binding of the KZG vector commitment, hence to n -SDH; history tamper-evidence reduces to collision resistance (Section 8).
- **A prototype.** A Python reference and a Rust/arkworks backend over BLS12-381 validate the design: eviction costs $\approx 135 \mu\text{s}$ independent of cache size up to 2^{16} , audit proofs are 48 bytes versus linearly growing Merkle multiproofs, and Feist–Khovratovich makes whole-cache proof generation practical (Section 9).

Scope. EvictVC is a commitment primitive, not an end-to-end verifiable-inference system. It takes the initial value and score commitments as given and certifies the trajectory that follows: which positions were removed, under which policy and threshold, carrying which values, and in what order. Whether $\mathbf{v}$ and $\mathbf{s}$ are in fact the key/value states and attention scores the model computed lies outside the primitive; establishing that is the task of the verifiable-inference layer surveyed in Section 2, with which EvictVC composes by sharing the commitment. The distinction bears on what an audit yields: an auditor who accepts every eviction in a trajectory learns that the compression followed the stated policy, not that the state being compressed was correct.

Organization. Section 2 surveys related work. Sections 3–5 develop the model. Sections 6–7 give the construction and Section 8 its security proofs. Section 9 reports the evaluation and Section 10 concludes.

2 Related Work

Verifiable and accountable LLM inference. The strongest guarantee is a zero-knowledge proof of the forward pass: zkLLM [SLZ24a] is the first zero-knowledge argument tailored to LLMs, proving an entire 13-billion-parameter inference in about fifteen minutes with a proof under 200 kB while keeping the model parameters hidden, via a custom lookup argument for the non-arithmetic tensor operations. Such proofs are sound but impose orders-of-magnitude prover overhead. A lighter line trades cryptographic soundness for empirical or economic guarantees: VeriLLM [W⁺25] commits to sampled activations with Merkle trees and re-executes roughly 1% of the computation under a one-honest-verifier assumption, deterring lazy verification with a peer-prediction incentive, while SVIP [SLZ⁺24b] and TopLoc [OFP⁺25] detect deviation through lightweight activation

Table 1: EvictVC against the vector-commitment lineage it builds on. The update mechanism is inherited; the precondition, the live/evicted distinction, and the committed record of the operation sequence are what the security model adds.

	VC [CF13]	UDA [CF13, §6.1]	EvictVC
Removing i	—	$\text{Update}(i, 1 \rightarrow 0)$	$\text{Evict}(i, \tau)$
Precondition	none	none	$\text{Pol}_\tau(s_i) = 1$, public
State afterwards	—	live, value 0	in E ; Verify rejects
Record kept	none	none	chain η , epoch t
Covered by security	position binding	set binding only	Defs. 7–9

fingerprints and locality-sensitive hashing. A feature common to all of these is that the prover commits to the *retained* computation—activations or transcripts that remain available to be opened or re-run. None models the *permanent deletion* of committed state, which is precisely what streaming cache compression performs and what EvictVC targets. Moreover, the Merkle-tree commitments several of these systems use yield audit proofs that grow with the number of checked positions, whereas EvictVC’s subvector audits are constant-size and aggregatable (Section 9).

Vector commitments. Vector commitments with position binding were formalized by Catalano and Fiore [CF13]; subsequent work added single-element subvector openings and cross-commitment aggregation [TAB⁺20] as well as efficient local updates. We reuse single-element subvector openings to make batch audits constant-size.

Removing a position by updating it to zero is not new, and we do not claim it. Catalano and Fiore already do exactly this: their universal dynamic accumulator from vector commitments realises Acc.Remove as an update of position i from 1 to 0 [CF13, §6.1]. What that construction does not provide is what EvictVC adds. Its removal carries no precondition—the accumulator’s secret key is the empty string, so any holder of the state can remove any element—it leaves no record of which removals occurred, and its security notions are silent about them: position binding quantifies over two openings at one position, and the accumulator’s set binding treats the removals requested through the oracle as defining the true set rather than asking whether each was permitted. Table 1 states the comparison.

KV-cache compression. Streaming cache compressors permanently evict the key/value states of low-importance tokens to bound memory. StreamingLLM [XTC⁺24] keeps “attention-sink” and recent tokens under a positional window; H2O [ZSZ⁺23] scores tokens by cumulative attention and retains the “heavy hitters”; SnapKV [LHY⁺24] selects clustered important tokens from a prefill observation window. In each case the unselected token states are removed from memory and become permanently inaccessible—the destructive operation our primitive must accommodate—and the threshold/score structure of these policies is exactly the policy predicate that EvictVC binds. These works optimize output quality and memory; they provide no verifiability, the orthogonal axis we add.

Amortized KZG proofs. The Feist–Khovratovich technique [FK23] computes all evaluation proofs of a KZG commitment in quasi-linear total time rather than recomputing each in isolation. We apply it (Section 9) so that opening proofs for the entire, possibly-evicted cache can be produced and maintained efficiently.

3 Preliminaries and Building Blocks

Notation. We write λ for the security parameter and 1^λ for its unary encoding. A function is *negligible*, written $\text{negl}(\lambda)$, if it vanishes faster than every inverse polynomial. PPT abbreviates probabilistic polynomial time. For $n \in \mathbb{N}$ we write $[n] = \{1, \dots, n\}$. We write $x \xleftarrow{\$} X$ for sampling x uniformly from a finite set X . All algorithms implicitly take pp as input once it is fixed. Concatenation of bit strings is denoted $\parallel$, and $\langle \cdot \rangle$ denotes an unambiguous (prefix-free) encoding into $\{0, 1\}^*$.

Bilinear groups. For the concrete instantiation we use a Type-III bilinear group $\text{bgp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, p)$ of prime order p , where $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a non-degenerate efficiently computable pairing. For $x \in \mathbb{F}_p$ we write $[x]_1 := x \cdot g_1$, $[x]_2 := x \cdot g_2$, $[x]_T := x \cdot e(g_1, g_2)$, so that $e([a]_1, [b]_2) = [ab]_T$. The abstract syntax of Section 4 is group-agnostic.

3.1 Vector commitments with updates

Definition 1 (Vector commitment). A *vector commitment* over message space $\mathcal{V}$ is a tuple of PPT algorithms $\text{VC} = (\text{VC.Setup}, \text{VC.Commit}, \text{VC.Open}, \text{VC.Verify}, \text{VC.Update})$:

- $\text{VC.Setup}(1^\lambda, 1^n) \rightarrow \text{pp}$ outputs public parameters for vectors of length n .
- $\text{VC.Commit}(\text{pp}, \mathbf{m}) \rightarrow (C, \text{st})$ commits to $\mathbf{m} \in \mathcal{V}^n$, returning a commitment C and committer state st .
- $\text{VC.Open}(\text{pp}, \text{st}, S) \rightarrow \pi$ produces a proof for the entries indexed by $S \subseteq [n]$.
- $\text{VC.Verify}(\text{pp}, C, S, \{m_i\}_{i \in S}, \pi) \rightarrow \{0, 1\}$ verifies the claimed entries against C .
- $\text{VC.Update}(\text{pp}, C, i, \delta) \rightarrow C'$ deterministically updates C to the commitment of the vector with m_i replaced by $m_i + \delta$.

We write $\text{Rem}(\text{pp}, C, i, m) := \text{VC.Update}(\text{pp}, C, i, -m)$ for the deterministic *removal* of value m from position i , and abbreviate $\text{VC.Verify}(\text{pp}, C, i, m, \pi)$ when $|S| = 1$.

Definition 2 (Position binding). VC is *position binding* if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}, \text{VC}}^{\text{pos-bind}}(\lambda) := \Pr \left[\begin{array}{l} \text{pp} \leftarrow \text{VC.Setup}(1^\lambda, 1^n); \\ (C, i, m, \pi, m', \pi') \leftarrow \mathcal{A}(\text{pp}) : \\ m \neq m' \wedge \text{VC.Verify}(\text{pp}, C, i, m, \pi) = 1 \\ \wedge \text{VC.Verify}(\text{pp}, C, i, m', \pi') = 1 \end{array} \right] = \text{negl}(\lambda).$$

3.2 Collision-resistant hashing

Definition 3 (CRHF). A hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is *collision resistant* if for every PPT adversary $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, H}^{\text{cr}}(\lambda) := \Pr[(x, x') \leftarrow \mathcal{A}(1^\lambda) : x \neq x' \wedge H(x) = H(x')] = \text{negl}(\lambda)$.

3.3 Eviction policies

Definition 4 (Eviction policy). Let $\mathcal{S}$ be a totally ordered *score space* and $\mathcal{T}$ a *threshold space*. An *eviction policy* is an efficiently computable predicate $\text{Pol} : \mathcal{T} \times \mathcal{S} \rightarrow \{0, 1\}$ that is *antitone* in the score: $s \leq s' \Rightarrow \text{Pol}(\tau, s) \geq \text{Pol}(\tau, s')$ for all τ . The canonical *threshold policy* is $\text{Pol}_{<}(\tau, s) = 1 \iff s < \tau$, modelling “evict only low-attention positions”. We write $\text{Pol}_\tau(s)$ for $\text{Pol}(\tau, s)$.

Remark 1. The threshold policy captures per-position eligibility, the regime our construction proves secure. The binding restriction is per-position rather than threshold: the definition admits any efficiently computable antitone predicate, and $\text{Pol}_{<}$ is named only as the canonical instance.

Set-relative policies reduce to the per-position case once the threshold may depend on the committed scores. Writing $s_{(1)} \leq \dots \leq s_{(m)}$ for the scores of the live positions in sorted order, evicting the k lowest is $\text{Pol}_{<}$ at $\tau = s_{(k+1)}$: the two agree exactly when the scores are distinct, and at a boundary tie $s_{(k)} = s_{(k+1)}$ the threshold form removes the positions strictly below, which is the part bottom- k leaves unspecified. What the reduction does not supply is a proof that the announced τ is that order statistic; in the syntax below the threshold is supplied to eviction verification rather than derived from the score commitment. Binding it would require committing to a sorted permutation of the score vector and proving both the permutation and the sortedness, or proving that exactly k live scores lie below τ —comparison or range arguments that KZG openings do not themselves provide. We treat set-relative policies as an extension (Section 10).

4 Evictable Vector Commitments

An *evictable vector commitment* (EvictVC) commits jointly to a value vector and a score vector, supports in-place value updates, and supports *eviction*: the permanent, policy-checked removal of a position together with an append-only, tamper-evident record of what was removed.

Public state. A commitment is a tuple $C = (C_{\text{val}}, C_{\text{sc}}, \eta, t)$, where C_{val} commits to the *live* value vector, C_{sc} commits to the score vector, $\eta \in \{0, 1\}^\lambda$ is the *eviction-history digest*, and $t \in \mathbb{N}$ is the *epoch* (number of evictions so far). The evicted set $E \subseteq [n]$ (with live set $\Lambda = [n] \setminus E$) is public metadata. A verifier initialises $E \leftarrow \emptyset$ at **Commit** and sets $E \leftarrow E \cup \{i\}$ on every eviction it has accepted, so E grows monotonically on the verifier's side and is never supplied by the prover. Committer state **st** holds the vectors, Λ , and t .

History digest. For an ordered eviction log $\mathcal{L} = (r_1, \dots, r_\ell)$ with records $r_k = \langle i_k, v_k, k-1 \rangle$, define the hash chain

$$\eta_0 = H(\langle \text{genesis} \rangle), \quad \eta_k = H(\eta_{k-1} \parallel \langle r_k \rangle), \quad \text{HistDig}(\mathcal{L}) := \eta_\ell.$$

4.1 Syntax

Definition 5 (EvictVC). An evictable vector commitment for value space $\mathcal{V}$, score space $\mathcal{S}$ and policy Pol is a tuple of PPT algorithms $\Pi = (\text{Setup}, \text{Commit}, \text{Open}, \text{Verify}, \text{Update}, \text{Evict}, \text{VerifyEvict})$:

Setup($1^\lambda, 1^n$) $\rightarrow$ **pp**. Outputs public parameters for vectors of length up to n .

Commit(**pp**, **v**, **s**) $\rightarrow$ (C , **st**). On $\mathbf{v} \in \mathcal{V}^n$, $\mathbf{s} \in \mathcal{S}^n$, outputs $C = (C_{\text{val}}, C_{\text{sc}}, \eta_0, 0)$ with $\Lambda = [n]$ and committer state **st**.

Open(**pp**, **st**, S) $\rightarrow$ π . For $S \subseteq \Lambda$, outputs a proof for the live values $\{v_i\}_{i \in S}$.

Verify(**pp**, C , E , S , $\{v_i\}_{i \in S}$, π) $\rightarrow$ $\{0, 1\}$. Outputs 1 iff $S \cap E = \emptyset$ and the claimed values verify against C_{val} .

Update(**pp**, **st**, i , δ) $\rightarrow$ (C' , **st'**). For $i \in \Lambda$, replaces v_i by $v_i + \delta$, sets $C'_{\text{val}} \leftarrow \text{VC.Update}(\text{pp}, C_{\text{val}}, i, \delta)$, leaving C_{sc}, η, t unchanged.

199 $\text{Evict}(\text{pp}, \text{st}, i, \tau) \rightarrow (C', \text{st}', \omega_i)$ or $\perp$. For $i \in \Lambda$: if $\text{Pol}_\tau(s_i) = 0$ output $\perp$; otherwise
 200 compute openings $\pi_i^{\text{val}}, \pi_i^{\text{sc}}$ of $C_{\text{val}}, C_{\text{sc}}$ at i ; set $C'_{\text{val}} \leftarrow \text{Rem}(\text{pp}, C_{\text{val}}, i, v_i)$, $C'_{\text{sc}} \leftarrow C_{\text{sc}}$,
 201 $\eta' \leftarrow H(\eta \parallel \langle i, v_i, t \rangle)$, $t' \leftarrow t + 1$, $\Lambda' \leftarrow \Lambda \setminus \{i\}$; output $C' = (C'_{\text{val}}, C'_{\text{sc}}, \eta', t')$, st' , and
 202 $\omega_i = (v_i, s_i, \pi_i^{\text{val}}, \pi_i^{\text{sc}})$.

203 $\text{VerifyEvict}(\text{pp}, C, i, \tau, \omega_i, C') \rightarrow \{0, 1\}$. Parse $\omega_i = (v, s, \pi^{\text{val}}, \pi^{\text{sc}})$. Output 1 iff: (a) $\text{VC.Verify}(\text{pp}, C_{\text{val}}, i, v, \pi^{\text{val}}) =$
 204 1 ; (b) $\text{VC.Verify}(\text{pp}, C_{\text{sc}}, i, s, \pi^{\text{sc}}) = 1$; (c) $\text{Pol}_\tau(s) = 1$; (d) $C'_{\text{val}} = \text{Rem}(\text{pp}, C_{\text{val}}, i, v)$ and
 205 $C'_{\text{sc}} = C_{\text{sc}}$; (e) $\eta' = H(\eta \parallel \langle i, v, t \rangle)$ and $t' = t + 1$.

206 *Remark 2* (Index consistency is structural). Checks (a)–(e) all refer to the *same* index i ,
 207 so an eviction of i cannot be justified by another position $j \neq i$'s score; this binding is
 208 syntactic and needs no computational assumption.

209 *Remark 3* (Why Verify consults E). Evict leaves C_{val} committing to a vector whose i -th
 210 entry is 0, so an opening of an evicted position to 0 satisfies VC.Verify . Consulting E
 211 separates “the live value is 0” from “the position is gone”. It also rules out reinstatement:
 212 a prover may add δL_i to C_{val} for $i \in E$, and since L_i vanishes at every other evaluation
 213 point this leaves openings at live positions unchanged, so no check on C_{val} alone detects
 214 it. The check $S \cap E = \emptyset$ rejects such positions, E being maintained by the verifier and
 215 monotone.

216 4.2 Correctness

217 **Definition 6** (Correctness). An execution is *admissible* if it is a **Commit** followed by
 218 any sequence of **Update**/**Evict** calls in which every call targets a currently live index and
 219 every **Evict** targets i with $\text{Pol}_\tau(s_i) = 1$ for the supplied τ . Π is *correct* if for all λ, n , all
 220 $(\mathbf{v}, \mathbf{s})$, and all admissible executions, with probability 1 over $\text{pp} \leftarrow \text{Setup}(1^\lambda, 1^n)$: (i) for
 221 every $S \subseteq \Lambda$, $\text{Verify}(\text{pp}, C, E, S, \{v_i\}_{i \in S}, \text{Open}(\text{pp}, \text{st}, S)) = 1$, and **Verify** outputs 0 on every
 222 S with $S \cap E \neq \emptyset$; and (ii) every $\text{Evict}(\text{pp}, \text{st}, i, \tau) \neq \perp$ producing $(C', \text{st}', \omega_i)$ satisfies
 223 $\text{VerifyEvict}(\text{pp}, C, i, \tau, \omega_i, C') = 1$.

224 5 Security Model

225 We require three properties, each an experiment between a challenger and a PPT adversary
 226 $\mathcal{A}$; a parsed commitment is $C = (C_{\text{val}}, C_{\text{sc}}, \eta, t)$ and $\text{Adv}(\lambda) := \Pr[\text{Exp}(\lambda) = 1]$.

227 5.1 Eviction soundness

228 A verifying eviction may only remove a position whose committed score satisfies the policy.

Experiment $\text{Exp}_{\mathcal{A}, \Pi}^{\text{ev-snd}}(\lambda)$:

1. $\text{pp} \leftarrow \text{Setup}(1^\lambda, 1^n)$.
- 229 2. $(C, i, \tau, \omega_i, C', s^\dagger, \pi^\dagger) \leftarrow \mathcal{A}(\text{pp})$, where $\omega_i = (\hat{v}, \hat{s}, \hat{\pi}^{\text{val}}, \hat{\pi}^{\text{sc}})$.
3. Output 1 iff (a) $\text{VerifyEvict}(\text{pp}, C, i, \tau, \omega_i, C') = 1$ and (b)
 $\text{VC.Verify}(\text{pp}, C_{\text{sc}}, i, s^\dagger, \pi^\dagger) = 1$ with $\text{Pol}_\tau(s^\dagger) = 0$.

230 **Definition 7** (Eviction soundness). Π is *eviction sound* if $\text{Adv}_{\mathcal{A}, \Pi}^{\text{ev-snd}}(\lambda) = \text{negl}(\lambda)$ for every
 231 PPT $\mathcal{A}$.

232 5.2 Policy-binding

233 The value an eviction removes (and records) is the unique committed value at that position.

Experiment $\text{Exp}_{\mathcal{A},\Pi}^{\text{pol-bind}}(\lambda)$:

1. $\text{pp} \leftarrow \text{Setup}(1^\lambda, 1^n)$.
2. $(C, i, \tau, \omega_i, C', v^\dagger, \pi^\dagger) \leftarrow \mathcal{A}(\text{pp})$, where $\omega_i = (\hat{v}, \hat{s}, \hat{\pi}^{\text{val}}, \hat{\pi}^{\text{sc}})$.
3. Output 1 iff (a) $\text{VerifyEvict}(\text{pp}, C, i, \tau, \omega_i, C') = 1$ and (b) $\text{VC.Verify}(\text{pp}, C_{\text{val}}, i, v^\dagger, \pi^\dagger) = 1$ with $v^\dagger \neq \hat{v}$.

Definition 8 (Policy-binding). Π is *policy-binding* if $\text{Adv}_{\mathcal{A},\Pi}^{\text{pol-bind}}(\lambda) = \text{negl}(\lambda)$ for every PPT $\mathcal{A}$.

5.3 History tamper-evidence

The eviction history is an append-only log whose digest commits to the full ordered sequence of evictions.

Experiment $\text{Exp}_{\mathcal{A},\Pi}^{\text{hist-te}}(\lambda)$:

1. $\text{pp} \leftarrow \text{Setup}(1^\lambda, 1^n)$.
2. $(\mathcal{L}, \mathcal{L}') \leftarrow \mathcal{A}(\text{pp})$, two eviction logs.
3. Output 1 iff $\mathcal{L} \neq \mathcal{L}'$ and $\text{HistDig}(\mathcal{L}) = \text{HistDig}(\mathcal{L}')$.

Definition 9 (History tamper-evidence). Π is *history tamper-evident* if $\text{Adv}_{\mathcal{A},\Pi}^{\text{hist-te}}(\lambda) = \text{negl}(\lambda)$ for every PPT $\mathcal{A}$.

Definition 10 (Secure EvictVC). Π is *secure* if it is correct (Definition 6) and satisfies eviction soundness (Definition 7), policy-binding (Definition 8), and history tamper-evidence (Definition 9).

Remark 4 (What the three properties jointly guarantee). Together they certify each eviction end-to-end: the removed position was policy-eligible under the committed scores (eviction soundness), the removed value was its genuine committed value and the live state is faithfully shrunk (policy-binding), and the cumulative record of removals is immutable (history tamper-evidence). An auditor holding C , the public evicted set E , and the per-step evidence can verify the entire compression trajectory without trusting the prover.

6 The KZG Vector Commitment

We instantiate the position-binding vector commitment of Definition 1 with the Lagrange-basis KZG commitment [KZG10] over a multiplicative subgroup of roots of unity. Throughout this and the next two sections we index positions by $\{0, \dots, n-1\}$, identifying position i with the domain point ω^i ; n is a power of two.

6.1 Hardness assumption

Security rests on the q -Strong Diffie–Hellman assumption of Boneh and Boyen [BB04], stated in *structured* (SRS) form so that the reductions of Section 8 can perfectly simulate the public parameters, which publish $\mathbb{G}_2$ powers for subvector verification.

Assumption 6.1 (n -SDH, SRS form). For $\tau \xleftarrow{\$} \mathbb{F}_p^*$ and every PPT $\mathcal{B}$,

$$\text{Adv}_{\mathcal{B}}^{n\text{-SDH}}(\lambda) := \Pr \left[\begin{array}{l} (c, W) \leftarrow \mathcal{B}(\{\tau^k\}_1^n, \{\tau^k\}_2^n) : \\ c \in \mathbb{F}_p \setminus \{\tau\} \wedge W = \left[\frac{1}{\tau - c} \right]_1 \end{array} \right] = \text{negl}(\lambda).$$

6.2 Construction

Let $\omega \in \mathbb{F}_p$ be a primitive n -th root of unity, $\mathbb{D} = \{\omega^i\}_{i=0}^{n-1}$, and $\{\ell_i\}_{i=0}^{n-1}$ the Lagrange basis for $\mathbb{D}$ ($\ell_i(\omega^j) = \delta_{ij}$, $\deg \ell_i < n$).

Construction 6.1 (KZG vector commitment VC_{KZG}).

VC.Setup($1^\lambda, 1^n$). Sample $\tau \xleftarrow{\$} \mathbb{F}_p^*$ and output $\text{pp} = (\{\tau^k\}_1^{n-1}, \{\tau^k\}_2^{n-1}, \{L_i\}_{i=0}^{n-1}, \omega)$ with $L_i := [\ell_i(\tau)]_1$. The trapdoor τ is erased. The $\mathbb{G}_2$ powers support subvector verification; single-point verification uses only $[1]_2, [\tau]_2$.

VC.Commit($\text{pp}, \mathbf{m}$). Treat $\mathbf{m} = (m_0, \dots, m_{n-1})$ as evaluations on $\mathbb{D}$; the interpolant is $\phi(X) = \sum_i m_i \ell_i(X)$. Output $C = [\phi(\tau)]_1 = \sum_i m_i L_i$ and $\text{st} = \mathbf{m}$.

VC.Open(pp, st, S). Let $Z_S(X) = \prod_{i \in S} (X - \omega^i)$ and r_S the interpolant of $\{(\omega^i, m_i)\}_{i \in S}$ ($\deg < |S|$). Output $\pi_S = [\psi_S(\tau)]_1$ where $\psi_S = (\phi - r_S)/Z_S$. For $S = \{i\}$, $\psi_i = (\phi - m_i)/(X - \omega^i)$.

VC.Verify($\text{pp}, C, S, \{m_i\}_{i \in S}, \pi_S$). Recompute r_S, Z_S and accept iff $e(C - [r_S(\tau)]_1, [1]_2) = e(\pi_S, [Z_S(\tau)]_2)$. For $S = \{i\}$ this is $e(C - [m_i]_1, [1]_2) = e(\pi_i, [\tau]_2 - \omega^i [1]_2)$.

VC.Update(pp, C, i, δ). Output $C + \delta L_i$; and $\text{Rem}(\text{pp}, C, i, m) := C - m L_i$.

Both $[r_S(\tau)]_1$ and $[Z_S(\tau)]_2$ are computable by the verifier from the SRS, S , and the claimed values, so verification needs no secret state.

6.3 Correctness of openings

Lemma 1 (Opening identity). *For honestly produced C, π_S , $\text{VC.Verify}(\text{pp}, C, S, \{m_i\}_{i \in S}, \pi_S) = 1$.*

Proof. By interpolation $\phi(\omega^i) = m_i = r_S(\omega^i)$ for $i \in S$, so $Z_S \mid (\phi - r_S)$ and $\phi - r_S = \psi_S Z_S$. Evaluating at the trapdoor and using bilinearity, $e(C - [r_S(\tau)]_1, [1]_2) = [\phi(\tau) - r_S(\tau)]_T = [\psi_S(\tau) Z_S(\tau)]_T = e(\pi_S, [Z_S(\tau)]_2)$. $\square$

Lemma 2 (Homomorphic update). *$\text{VC.Update}(\text{pp}, C, i, \delta)$ is the commitment to $\mathbf{m}$ with m_i replaced by $m_i + \delta$, and $\text{Rem}(\text{pp}, C, i, m_i)$ is the commitment with m_i replaced by 0.*

Proof. $C + \delta L_i = [(\phi + \delta \ell_i)(\tau)]_1$, and $\phi + \delta \ell_i$ interpolates $\mathbf{m}$ with the i -th evaluation shifted by δ since $\ell_i(\omega^j) = \delta_{ij}$. Removal is $\delta = -m_i$. $\square$

Remark 5 (Position binding). VC_{KZG} is position binding under n -SDH; the reduction is Lemma 3.

7 The EvictVC Construction

We build Π_{KZG} by committing to the value and score vectors with two instances of VC_{KZG} under a shared pp , and binding the history with a hash chain. We write $\text{VC.Verify}_{\text{val}}$ and $\text{VC.Verify}_{\text{sc}}$ for verification against C_{val} and C_{sc} ; both are Construction 6.1's algorithm.

Construction 7.1 (Π_{KZG}). Fix a collision-resistant $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ and policy Pol .

Setup($1^\lambda, 1^n$). Output $\text{pp} \leftarrow \text{VC.Setup}(1^\lambda, 1^n)$.

Commit($\text{pp}, \mathbf{v}, \mathbf{s}$). Set $C_{\text{val}} \leftarrow \text{VC.Commit}(\text{pp}, \mathbf{v})$, $C_{\text{sc}} \leftarrow \text{VC.Commit}(\text{pp}, \mathbf{s})$, $\eta_0 \leftarrow H(\langle \text{genesis} \rangle)$, $t \leftarrow 0$, $\Lambda \leftarrow \{0, \dots, n-1\}$; output $C = (C_{\text{val}}, C_{\text{sc}}, \eta_0, 0)$, st .

300 **Open/Verify.** VC.Open on C_{val} ; **Verify** rejects if $S \cap E \neq \emptyset$, otherwise runs $\text{VC.Verify}_{\text{val}}$ on
 301 C_{val} .

302 **Update**($\text{pp}, \text{st}, i, \delta$). $C'_{\text{val}} \leftarrow \text{VC.Update}(\text{pp}, C_{\text{val}}, i, \delta)$, $v_i += \delta$.

303 **Evict**($\text{pp}, \text{st}, i, \tau_p$). If $\text{Pol}_{\tau_p}(s_i) = 0$ output $\perp$; else compute $\pi_i^{\text{val}} \leftarrow \text{VC.Open}(\text{pp}, \mathbf{v}, \{i\})$,
 304 $\pi_i^{\text{sc}} \leftarrow \text{VC.Open}(\text{pp}, \mathbf{s}, \{i\})$; set $C'_{\text{val}} \leftarrow C_{\text{val}} - v_i L_i$, $\eta' \leftarrow \text{H}(\eta \parallel \langle i, v_i, t \rangle)$, $t' \leftarrow t + 1$,
 305 $\Lambda' \leftarrow \Lambda \setminus \{i\}$; output $C' = (C'_{\text{val}}, C'_{\text{sc}}, \eta', t')$, st' , $\omega_i = (v_i, s_i, \pi_i^{\text{val}}, \pi_i^{\text{sc}})$.

306 **VerifyEvict**($\text{pp}, C, i, \tau_p, \omega_i, C'$). Parse $\omega_i = (v, s, \pi^{\text{val}}, \pi^{\text{sc}})$; output 1 iff (a) $\text{VC.Verify}_{\text{val}}(\text{pp}, C_{\text{val}}, i, v, \pi^{\text{val}}) =$
 307 1 ; (b) $\text{VC.Verify}_{\text{sc}}(\text{pp}, C_{\text{sc}}, i, s, \pi^{\text{sc}}) = 1$; (c) $\text{Pol}_{\tau_p}(s) = 1$; (d) $C'_{\text{val}} = C_{\text{val}} - v L_i$ and $C'_{\text{sc}} = C_{\text{sc}}$;
 308 (e) $\eta' = \text{H}(\eta \parallel \langle i, v, t \rangle)$ and $t' = t + 1$.

309 The eviction threshold is written τ_p to distinguish it from the SRS trapdoor τ .

310 **Theorem 1** (Correctness). Π_{KZG} satisfies Definition 6.

311 *Proof.* (i) At every state $C_{\text{val}} = [\phi(\tau)]_1$ for the interpolant ϕ of the current value vector:
 312 **Commit** establishes this, **Update** preserves it and **Evict** zeroes position i , both by Lemma 2;
 313 hence any $S \subseteq \Lambda$ verifies by Lemma 1, and **Verify** rejects any S meeting E before it consults
 314 C_{val} . (ii) If **Evict** $\neq \perp$ then $\text{Pol}_{\tau_p}(s_i) = 1$; checks (a),(b) hold by Lemma 1, (c) by eligibility,
 315 (d) since $C'_{\text{val}} = C_{\text{val}} - v_i L_i$ and C_{sc} is unchanged, and (e) since η', t' were produced by
 316 the same step **VerifyEvict** recomputes. $\square$

317 **Remark 6** (One-element batch audits). A verifier auditing any set S of live positions calls
 318 $\text{VC.Verify}_{\text{val}}$ on the single subvector proof $\pi_S = [\psi_S(\tau)]_1$: one $\mathbb{G}_1$ element, verified with
 319 two pairings, independent of $|S|$. This is the constant-size audit proof that distinguishes
 320 the scheme from Merkle multiproofs.

321 **Remark 7** (Amortized opening). Naively one opening costs $O(n \log n)$. By Feist–Khovratovich [FK23],
 322 all n single-point proofs are produced by a single $O(n \log n)$ group-FFT computation, i.e.
 323 $O(\log n)$ amortized, and are maintainable under updates in the same complexity.

324 The asymptotic costs of Π_{KZG} (n = vector length; $\mathbb{G}_1$ operations unless noted):

Operation	Time	Output / proof size
Setup (one-time)	$O(n)$ scalar mults + $O(n \log n)$ for $\{L_i\}$	pp of size $O(n)$
Commit	$O(n)$ (MSM)	one $\mathbb{G}_1$ element
326 Update / Evict (commit.)	$O(1)$	one $\mathbb{G}_1$ element
history step	$O(1)$ (one hash)	λ bits
Open (single / subvector)	$O(n \log n)$; $O(\log n)$ amort. (FK)	one $\mathbb{G}_1$ element
Verify / VerifyEvict	$O(1)$ (two pairings)	—

327 8 Security Proofs

328 The two binding-type properties reduce to position binding of VC_{KZG} under n -SDH; history
 329 tamper-evidence reduces to collision resistance. We write $\mathcal{B}$ for the reduction adversary.

330 8.1 Position binding of the KZG vector commitment

331 **Lemma 3** (Position binding from n -SDH). For every PPT $\mathcal{A}$ against position binding of
 332 VC_{KZG} (Definition 2) there is a PPT $\mathcal{B}$ with $\text{Adv}_{\mathcal{A}, \text{VC}_{\text{KZG}}}^{\text{pos-bind}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{n\text{-SDH}}(\lambda) + n/p$.

333 *Proof.* $\mathcal{B}$ receives an SRS-form n -SDH challenge $(\{[\tau^k]_1\}_{k=0}^n, \{[\tau^k]_2\}_{k=0}^n)$ and assembles
 334 $\mathbf{pp}$ as in Construction 6.1: it keeps the powers up to τ^{n-1} , fixes ω , and computes each
 335 $L_i = \sum_k c_{i,k} [\tau^k]_1$ from the public coefficients $\ell_i(X) = \sum_k c_{i,k} X^k$. This $\mathbf{pp}$ is distributed
 336 as in the real experiment, so $\mathcal{B}$ runs $\mathcal{A}(\mathbf{pp}) \rightarrow (C, i, m, \pi, m', \pi')$.

337 Suppose $\mathcal{A}$ wins: $m \neq m'$ and, with $z := \omega^i$,

$$338 \quad e(C - [m]_1, [1]_2) = e(\pi, [\tau]_2 - z[1]_2), \quad (1)$$

$$339 \quad e(C - [m']_1, [1]_2) = e(\pi', [\tau]_2 - z[1]_2). \quad (2)$$

340 Dividing (1) by (2), the C -terms cancel and $e([m' - m]_1, [1]_2) = e(\pi - \pi', [\tau - z]_2)$. If
 341 $z = \tau$ the right side is the identity, forcing $m = m'$; hence $z \neq \tau$, an event whose
 342 complement we condition on (since $z \in \mathbb{D}$ and $\tau \xleftarrow{\$} \mathbb{F}_p^*$, $\tau \in \mathbb{D}$ has probability $\leq n/p$).
 343 With $\Delta := m' - m \neq 0$ and $W := \Delta^{-1}(\pi - \pi')$,

$$344 \quad e(W, [\tau - z]_2) = e(\pi - \pi', [\tau - z]_2)^{\Delta^{-1}} = e([\Delta]_1, [1]_2)^{\Delta^{-1}} = [1]_T = e(g_1, g_2),$$

345 so by non-degeneracy $W = [1/(\tau - z)]_1$, and $\mathcal{B}$ outputs (z, W) . $\square$

346 8.2 Eviction soundness

347 **Theorem 2** (Eviction soundness). *For every PPT $\mathcal{A}$ there is a PPT $\mathcal{B}$ with $\text{Adv}_{\mathcal{A}, \Pi_{\text{KZG}}}^{\text{ev-snd}}(\lambda) \leq$*
 348 $\text{Adv}_{\mathcal{B}, \text{VC}_{\text{KZG}}}^{\text{pos-bind}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{n\text{-SDH}}(\lambda) + n/p$.

349 *Proof.* $\mathcal{B}$ forwards $\mathbf{pp}$ to $\mathcal{A}$ and runs the eviction-soundness experiment, obtaining $(C, i, \tau_p, \omega_i, C', s^\dagger, \pi^\dagger)$
 350 with $\omega_i = (\hat{v}, \hat{s}, \hat{\pi}^{\text{val}}, \hat{\pi}^{\text{sc}})$. If $\mathcal{A}$ wins, VerifyEvict accepts, so by checks (b),(c) $\text{VC.Verify}(\mathbf{pp}, C_{\text{sc}}, i, \hat{s}, \hat{\pi}^{\text{sc}}) =$
 351 1 with $\text{Pol}_{\tau_p}(\hat{s}) = 1$, while the winning condition gives $\text{VC.Verify}(\mathbf{pp}, C_{\text{sc}}, i, s^\dagger, \pi^\dagger) = 1$ with
 352 $\text{Pol}_{\tau_p}(s^\dagger) = 0$. As $\text{Pol}_{\tau_p}(\hat{s}) \neq \text{Pol}_{\tau_p}(s^\dagger)$, $\hat{s} \neq s^\dagger$, so $(C_{\text{sc}}, i, \hat{s}, \hat{\pi}^{\text{sc}}, s^\dagger, \pi^\dagger)$ breaks position
 353 binding on C_{sc} . Apply Lemma 3. $\square$

354 8.3 Policy-binding

355 **Theorem 3** (Policy-binding). *For every PPT $\mathcal{A}$ there is a PPT $\mathcal{B}$ with $\text{Adv}_{\mathcal{A}, \Pi_{\text{KZG}}}^{\text{pol-bind}}(\lambda) \leq$*
 356 $\text{Adv}_{\mathcal{B}, \text{VC}_{\text{KZG}}}^{\text{pos-bind}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{n\text{-SDH}}(\lambda) + n/p$.

357 *Proof.* $\mathcal{B}$ runs $\mathcal{A}$ and obtains $(C, i, \tau_p, \omega_i, C', v^\dagger, \pi^\dagger)$ with $\omega_i = (\hat{v}, \hat{s}, \hat{\pi}^{\text{val}}, \hat{\pi}^{\text{sc}})$. If $\mathcal{A}$ wins,
 358 VerifyEvict accepts, so by check (a) $\text{VC.Verify}(\mathbf{pp}, C_{\text{val}}, i, \hat{v}, \hat{\pi}^{\text{val}}) = 1$, while the winning
 359 condition gives $\text{VC.Verify}(\mathbf{pp}, C_{\text{val}}, i, v^\dagger, \pi^\dagger) = 1$ with $v^\dagger \neq \hat{v}$. Thus $(C_{\text{val}}, i, \hat{v}, \hat{\pi}^{\text{val}}, v^\dagger, \pi^\dagger)$
 360 breaks position binding on C_{val} . Apply Lemma 3. $\square$

361 *Remark 8.* Checks (a) and (d) together pin down the post-eviction state: $\hat{v}$ is the unique
 362 committed value at i and $C'_{\text{val}} = C_{\text{val}} - \hat{v}L_i$, so position i holds zero in C'_{val} . The checks
 363 cover that step alone; a commitment published later may differ from C'_{val} at i , and what
 364 carries irreversibility across the trajectory is E together with the liveness check in Verify
 365 (Remark 3).

366 8.4 History tamper-evidence

367 **Theorem 4** (History tamper-evidence). *If H is collision resistant then for every PPT $\mathcal{A}$*
 368 *there is a PPT $\mathcal{B}$ with $\text{Adv}_{\mathcal{A}, \Pi_{\text{KZG}}}^{\text{hist-te}}(\lambda) \leq \text{Adv}_{\mathcal{B}, H}^{\text{cr}}(\lambda)$.*

369 *Proof.* $\mathcal{B}$ runs $\mathcal{A}$ to get $\mathcal{L} = (r_1, \dots, r_\ell)$, $\mathcal{L}' = (r'_1, \dots, r'_{\ell'})$ with $\mathcal{L} \neq \mathcal{L}'$ and equal digest, and
 370 walks the two chains backward maintaining “current digests equal”. At a step comparing
 371 $\eta_a = \eta'_b$: if $a, b \geq 1$ then $H(\eta_{a-1} \parallel \langle r_a \rangle) = H(\eta'_{b-1} \parallel \langle r'_b \rangle)$; if the pre-images differ, output
 372 them as a collision, else (fixed-length chaining value and prefix-free $\langle \cdot \rangle$) $\eta_{a-1} = \eta'_{b-1}$,

373 $r_a = r'_b$, and recurse. If exactly one of a, b is 0, the two pre-images differ by domain
 374 separation of the genesis tag, yielding a collision. If $a = b = 0$, every record matched, so
 375 $\mathcal{L} = \mathcal{L}'$, contradicting $\mathcal{L} \neq \mathcal{L}'$. Hence a collision is always emitted. $\square$

376 8.5 Main theorem

377 **Theorem 5** (Security of Π_{KZG}). *If n -SDH holds and H is collision resistant, then Π_{KZG} is a*
 378 *secure evictable vector commitment (Definition 10), with $\text{Adv}^{\text{ev-snd}}, \text{Adv}^{\text{pol-bind}} \leq \text{Adv}_{\mathcal{B}}^{n\text{-SDH}} +$*
 379 *n/p and $\text{Adv}^{\text{hist-te}} \leq \text{Adv}_{\mathcal{B}, H}^{\text{cr}}$.*

380 *Proof.* Correctness is Theorem 1; the three properties are Theorems 2, 3, 4, negligible
 381 under n -SDH (via Lemma 3, with n/p negligible for λ -bit p) and collision resistance
 382 respectively. $\square$

383 *Remark 9* (Tightness and instantiation). All reductions invoke their adversary once and
 384 preserve advantage up to the additive n/p from the trapdoor-collision event, $\leq 2^{-\lambda + \log n}$
 385 for $p \approx 2^\lambda$. Over BLS12-381 (255-bit p , 256-bit H) the bounds are below 2^{-128} for all
 386 $n \leq 2^{40}$.

387 9 Evaluation

388 **Implementation.** We implement Π_{KZG} twice: a Python reference (`py_ecc`, BLS12-381)
 389 covering the full eviction protocol—policy-checked eviction, tamper-evident hash-chain history,
 390 and single-element subvector audits—and a performance backend in Rust on `arkworks`
 391 (`ark-bls12-381`). The Rust backend realizes commitment, $O(1)$ update/eviction against
 392 precomputed Lagrange-basis commitments, single-point opening and pairing verification,
 393 and the Feist–Khovratovich all-proofs routine.

394 **Setup.** Measurements are taken single-threaded on an Apple M-series laptop with a
 395 Rust release build, for vector (cache) lengths n from 2^{10} to 2^{16} .

396 9.1 Commitment maintenance

397 Table 2 reports per-operation cost as the cache length n ranges over a $64\times$ window. The
 398 defining property of the scheme is visible directly in the *update* column: eviction and value
 399 update cost $127\text{--}142\mu\text{s}$ with no upward trend, the variation being measurement noise
 400 rather than growth in n . This is forced by the construction. An update is $C'_{\text{val}} = C_{\text{val}} \pm \delta L_i$:
 401 one scalar multiplication and one point addition against the precomputed Lagrange-basis
 402 commitment L_i , touching a single group element and never reading or recombining the
 403 other $n - 1$ positions. The cost is therefore independent of n by construction, and an
 404 eviction—a removal $C_{\text{val}} - v L_i$ plus one hash for the history step—inherits the same
 405 $O(1)$ profile. For comparison, recommitting from scratch is $O(n)$ and even an incremental
 406 Merkle update is $O(\log n)$; only the homomorphic group update is genuinely constant-time,
 407 which is what makes per-token eviction viable in a streaming setting where the cache is
 408 recompressed continuously.

409 Verification is likewise flat (2.57–2.67 ms): it is two pairings whose cost is fixed by the
 410 curve, not by the cache, so an audit of any single position—or, by Remark 6, of any *set*
 411 of positions—verifies in constant time. Commitment and opening grow roughly linearly
 412 (about $3.3\text{--}3.4\times$ per $4\times$ increase in n), reflecting the multi-scalar multiplication that
 413 dominates the $O(n \log n)$ interpolation transform at these sizes. **Setup** is a one-time $O(n)$
 414 cost; in deployment it is a powers-of-tau ceremony artifact loaded once and never appears
 415 on the per-operation path. The measurements are single-threaded; what transfers across

Table 2: Per-operation cost of Π_{KZG} (BLS12-381, single thread). Eviction/update is $O(1)$ and verification is constant; commit/open are $O(n \log n)$ and setup is a one-time $O(n)$ cost.

n	Setup (ms)	Commit (ms)	update (μs)	Open (ms)	Verify (ms)
2^{10}	167.3	18.2	141.5	18.3	2.57
2^{12}	622.4	59.2	126.8	59.2	2.57
2^{14}	2487.6	200.8	135.9	200.5	2.60
2^{16}	9954.7	681.0	135.9	683.8	2.67

hardware is the *shape*—flat update, constant verification, near-linear commit/open—not the absolute constants.

9.2 Audit proof size

Figure 1 plots batch-audit proof size against the number of audited positions k at a fixed cache of $n = 2^{14}$. The EvictVC curve is a horizontal line at 48 bytes: a batch audit of any set S is the single subvector opening $\pi_S = [\psi_S(\tau)]_1$, and $\psi_S = (\phi - r_S)/Z_S$ is one polynomial whose commitment is one $\mathbb{G}_1$ element—a compressed BLS12-381 point—no matter how many positions S contains or how large n is. The Merkle baseline, by contrast, carries one authentication path of $\lceil \log_2 n \rceil$ sibling hashes (32 bytes each) per audited leaf: even a single-position proof is 448 bytes at $n = 2^{14}$ (14 nodes), already $9\times$ the KZG proof, and a k -position multiproof grows with k until it reaches hundreds of kilobytes.

The Merkle curve rises and then falls as $k \rightarrow n$: when nearly all leaves are audited the paths share most internal nodes, so the number of *distinct* authentication nodes shrinks (auditing all n leaves needs none—the auditor holds the whole tree). This is a correct artifact of multiproof deduplication; the operative comparison is the realistic regime $k \ll n$, where a verifier spot-checks a sample of cached positions. There Merkle is essentially linear in k while EvictVC is constant, a one- to three-order-of-magnitude reduction over the plotted range. Aggregation also collapses verification: the k -position audit is still two pairings, against the Merkle verifier’s $O(k \log n)$ hashing. We stress that the advantage is in proof size and aggregation, not single-update latency—a Merkle leaf update is a handful of hashes and is in fact cheaper than one group operation; EvictVC’s case rests on constant-size, aggregatable audits and the $O(1)$ commitment maintenance of Table 2, not on beating a hash with a pairing.

9.3 Amortized proof generation

Table 3 contrasts the Feist–Khovratovich all-proofs routine with the naive baseline of computing one opening per position. The two scale differently in the most consequential way. Naively, a single opening costs $O(n \log n)$, so producing a proof for every position costs $O(n^2 \log n)$; the “naive all” column is simply n times the measured single-open cost. Feist–Khovratovich instead obtains all n proofs from one $O(n \log n)$ group-FFT pass, an $O(\log n)$ amortized cost per position. This shows up in the table as a qualitative difference in the per-proof trend: FK’s amortized cost creeps only from 3.1 to 4.7 ms across 2^8 to 2^{12} (a logarithmic drift), while the naive single-open cost climbs roughly linearly (6 to 59 ms over the same window). The crossover is immediate—at every measured size FK’s amortized per-proof already undercuts a single naive opening (at 2^{12} , 4.7 ms versus 59 ms, a $12\times$ gap)—and the ratio widens with n .

At full-cache scale the separation becomes decisive. A single opening at $n = 2^{16}$ costs ≈ 684 ms (Table 2), so producing all 2^{16} proofs naively would take $684 \text{ ms} \times 2^{16} \approx 4.5 \times 10^4 \text{ s}$,

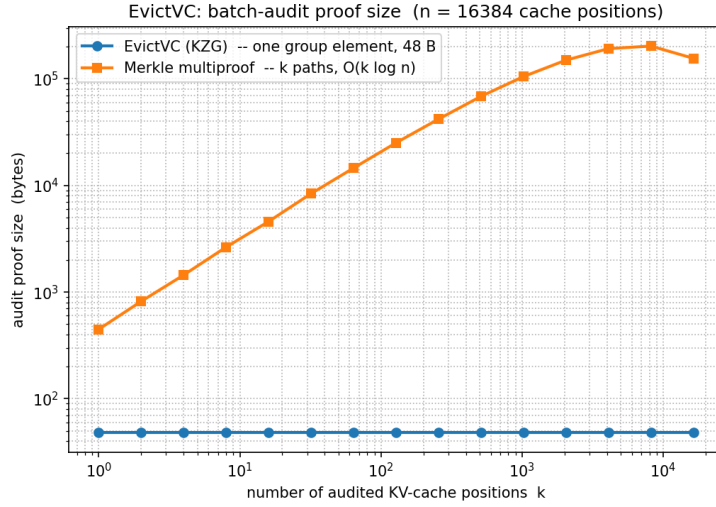


Figure 1: Batch-audit proof size versus number of audited positions k ($n = 2^{14}$). EvictVC’s proof is a constant 48-byte group element; the Merkle multiproof grows with k .

Table 3: Feist–Khovratovich all-proofs versus naive per-position opening (BLS12-381, single thread). FK’s amortized per-proof cost grows only logarithmically; the naive all-proofs estimate is $n \times$ a single opening.

n	FK all (ms)	FK / proof (μ s)	naive open (ms)	naive all (est., s)
2^8	800.7	3127.8	6.13	1.6
2^{10}	3980.3	3887.0	18.32	18.8
2^{12}	19095.5	4662.0	59.21	242.5

about 12.5 hours; the FK routine, extrapolating its $O(n \log n)$ trend, completes the same task in roughly 7 minutes single-threaded, and its FFT parallelizes cleanly across cores. Beyond a handful of openings, FK is therefore not merely faster but the only feasible way to serve whole-cache audits. One honest caveat: FK’s absolute per-proof cost is milliseconds, not microseconds, because the group-FFT performs $O(n \log n)$ scalar multiplications by roots of unity; the win is the asymptotic $O(n \log n)$ -versus- $O(n^2)$ separation in total work—realized here as hours versus minutes—rather than a small per-proof constant.

Summary. The three measurements validate the design end-to-end and along independent axes. Constant-time eviction (Table 2) makes it practical to apply the commitment per evicted token in a streaming compressor, where the cache is rewritten continuously. Constant-size, aggregatable audits (Figure 1) make verification scale *in* the number of checked positions rather than against it, so an auditor can sample as aggressively as it likes for a fixed 48-byte proof and two pairings. And amortized proof generation (Table 3) makes serving openings for the entire, possibly-evicted cache feasible at a scale where the naive approach is hours out of reach. None of these rests on a faster primitive than the alternatives in isolation; each follows from the algebraic structure—homomorphic $O(1)$ updates, one-element subvector openings, and batched proof computation—that the KZG instantiation provides and a hash-based commitment does not.

10 Conclusion

We introduced evictable vector commitments, a primitive that brings permanent, policy-checked deletion—with a tamper-evident history—to the vector commitments underlying verifiable LLM inference, and gave a KZG construction with $O(1)$ eviction, single-element batch-audit proofs, and quasi-linear all-proof maintenance, proven secure under n -SDH and collision resistance.

Several extensions are natural. *Set-relative policies* such as arg min/bottom- k eviction require a verifiable minimality or sorting argument over the committed score vector, layered on the threshold policy proven here. *Succinct history membership* would let an auditor prove a single past eviction without replaying the whole log, e.g. by replacing the hash chain with a Merkle Mountain Range. *Confidential eviction* could hide the evicted values—revealing only their commitments and update proofs—when the dropped state is sensitive. Finally, integrating EvictVC into an end-to-end verifiable-inference pipeline over a real streaming KV-cache compressor is the natural systems follow-up.

References

- [BB04] Dan Boneh and Xavier Boyen. Short signatures without random oracles. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 56–73. Springer, Heidelberg, May 2004. doi:10.1007/978-3-540-24676-3_4.
- [CF13] Dario Catalano and Dario Fiore. Vector commitments and their applications. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013*, volume 7778 of *LNCS*, pages 55–72. Springer, Heidelberg, February / March 2013. doi:10.1007/978-3-642-36362-7_5.
- [FK23] Dankrad Feist and Dmitry Khovratovich. Fast amortized KZG proofs. Cryptology ePrint Archive, Report 2023/033, 2023. <https://eprint.iacr.org/2023/033>.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Heidelberg, December 2010. doi:10.1007/978-3-642-17373-8_11.
- [LHY⁺24] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 22947–22970, 2024.
- [OFP⁺25] Jack Min Ong, Matthew Di Ferrante, Aaron Pazdera, Ryan Garner, Sami Jaghouar, Manveer Basra, Max Ryabinin, and Johannes Hagemann. TOPLOC: A locality-sensitive hashing scheme for trustless verifiable inference. In *42nd International Conference on Machine Learning (ICML 2025)*, volume 267 of *Proceedings of Machine Learning Research*, 2025. doi:10.48550/arXiv.2501.16007.
- [SLZ24a] Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models. In *ACM CCS 2024*, pages 4405–4419. ACM Press, 2024. doi:10.1145/3658644.3670334.
- [SLZ⁺24b] Yifan Sun, Yuhang Li, Yue Zhang, Yuchen Jin, and Huan Zhang. SVIP: Towards verifiable inference of open-source large language models. arXiv:2410.22307, 2024. doi:10.48550/arXiv.2410.22307.

- 517 [TAB⁺20] Alin Tomescu, Ittai Abraham, Vitalik Buterin, Justin Drake, Dankrad Feist,
518 and Dmitry Khovratovich. Aggregatable subvector commitments for stateless
519 cryptocurrencies. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN*
520 *20*, volume 12238 of *LNCS*, pages 45–64. Springer, Heidelberg, September 2020.
521 [doi:10.1007/978-3-030-57990-6_3](https://doi.org/10.1007/978-3-030-57990-6_3).
- 522 [W⁺25] Ke Wang et al. VeriLLM: A lightweight framework for publicly verifiable
523 decentralized inference. arXiv:2509.24257, 2025. [doi:10.48550/arXiv.2509.](https://doi.org/10.48550/arXiv.2509.24257)
524 [24257](https://doi.org/10.48550/arXiv.2509.24257).
- 525 [XTC⁺24] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis.
526 Efficient streaming language models with attention sinks. In *12th International*
527 *Conference on Learning Representations (ICLR 2024)*, 2024. [doi:10.48550/a](https://doi.org/10.48550/arXiv.2309.17453)
528 [rXiv.2309.17453](https://doi.org/10.48550/arXiv.2309.17453).
- 529 [ZSZ⁺23] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi
530 Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang
531 Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative infer-
532 ence of large language models. In *Advances in Neural Information Processing*
533 *Systems 36 (NeurIPS 2023)*, pages 34661–34710, 2023.